CSCI 4100

Operating Systems and Architecture

Chapter 5: Synchronizing Access to Shared Objects

Topics

- 5.0 – Synchronizing Access to Shared Objects
- 5.1 – Challenges
- 5.2 – Structuring Shared Objects
- 5.3 – Locks: Mutual Exclusion
- 5.4 – Condition Variables: Waiting for a Change
- 5.5 – Designing and Implementing Shared Objects
- 5.6 – Three Case Studies
- 5.7 – Implementing Synchronization Objects
- 5.8 – Semaphores Considered Harmful

Synchronizing Access to Shared Objects

- **Independent threads** operate on separate subsets of memory.
- **Cooperating threads** share state, creating new challenges.
  - Different orderings of operations on shared state can lead to different results.
  - Execution is nondeterministic, resulting in bugs that are hard to reproduce.
  - Compilers and processors can reorder instructions to improve performance.
- A **structured synchronization** approach can prevent these problems.
  - Structure programs to make it easier to reason about concurrency.
  - Use standard synchronization primitives to control access to shared state.

Race Conditions – Example 1

A **race condition** occurs when the behavior of a program depends on the interleaving of operations of different threads.

Example 1:

- Assume two threads, A and B, and a shared variable `x`.
- Thread A code: `x = 1;`
- Thread B code: `x = 2;`
- Depending on which thread executes last, `x` might be `1` or `2`.

Race Conditions – Example 2

Example 2:

- Assume a shared variable `y` with an initial value of `12`.
- Assume also that loading and storing values are **atomic instructions** that can not be interrupted.
- Thread A code: `x = y + 1;`
- Thread B code: `y = y * 2;`
- If thread A loads `y` before thread B stores `y`, the value of `x` is `13`.
- If thread B stores `y` before thread A loads `y`, the value of `x` is `25`.

Race Conditions – Example 3

Example 3:

- Assume a shared variable `x` with an initial value of `0`.
- Thread A code: `x = x + 1;`
- Thread B code: `x = x + 2;`
- Consider three different cases:
  1. Thread A completes before thread B begins: the value of `x` is `3`.
  2. Threads A and B both load `x` before either stores `x`, and thread A stores before thread B: the value of `x` is `2`.
  3. Same as case 2, except thread B stores before thread A: the value of `x` is `1`.

Too Much Milk

Consider the following scenario:

- Two roommates (R1 and R2) check the fridge at different times and notice they are out of milk.
- Both roommates independently leave to purchase more milk.
- As a result they end up with too much milk in the fridge.

This problem can be solved by ensuring the following conditions:

- **Safety**: At most one person buys milk.
- **Liveness**: If milk is needed, someone eventually buys it.

Too Much Milk – Solution 1

Solution 1: Leave a note on the fridge before going to the store.

- Scenario A:
  1. R1 leaves a note.
  2. R2 sees the note.
  3. Only R1 buys milk.
- Scenario B:
  1. R1 checks the fridge.
  2. R2 checks the fridge.
  3. R1 leaves a note.
  4. R2 leaves a note.
  5. Both roommates go buy milk, which violates the safety condition.

Too Much Milk – Solution 2

Solution 2: After leaving a note, a roommate checks if the other roommate left a note before going to the store.

- Since a roommate can not buy milk unless they have written a note and the other roommate has not, the safety condition holds.
- Both roommates could leave a note before the other roommate checks for a note, violating liveness.

Too Much Milk – Solution 3

Solution 3: Make sure one roommate has bought milk before the other one decides not to buy milk.

- If R1 leaves a note, then sees that R2 has left a note, R1 waits for R2 to remove the note before checking if there is any milk in the fridge.
- R2 uses the procedure from solution 2.
- This approach guarantees liveness and safety, but is inefficient because R1 uses a busy wait, repeatedly checking if R2 has removed the note.

Too Much Milk – Solution 4

Solution 4: Use a synchronization primitive.

- A roommate must gain exclusive access to the fridge before checking if there is any milk.
- That access is only relinquished after milk has been purchased.
- A roommate waiting for access does not need to use a busy wait, instead the relinquishing roommate signals to the waiting roommate that they may now access the fridge.

Structuring Shared Objects

Shared objects are objects that can be accessed safely by multiple threads.

Shared objects are implemented in three layers:

1. The **shared object layer** provides an interface for accessing and modifying the shared object.
2. The **synchronization variable layer** provides synchronization variables, like locks and condition variables, used to guard state variables that contain the object's data.
3. The **atomic instruction layer** provides processor-specific instructions that read, modify, and write data atomically. These instructions are used to implement synchronization variables.

Locks: Mutual Exclusion

A **lock** is a synchronization variable that provides **mutual exclusion**.

- When one thread holds a lock, no other thread can hold it.
- A thread can make a sequence of instructions appear to be one atomic **critical section** by holding the lock while executing those instructions.
- Each shared object usually has one lock guarding the state of that object.
- Locks allow a programmer to establish **invariants**, or properties that must hold before and after changes to the state of the object.

Lock Operations

A lock provides two operations: `acquire` and `release`.

- A lock can be in a BUSY or FREE state. Initially it is FREE.
- `acquire` waits until the lock is FREE, then atomically makes the lock BUSY.
- If multiple threads call `acquire` at once, only one will be successful.
- `release` makes the lock FREE and allows at most one thread waiting for the lock to acquire it.

Lock Properties

A lock should ensure the following properties:

- **Mutual Exclusion**: At most one thread can hold the lock.
- **Progress**: If no thread holds the lock, then some thread can successfully acquire the lock.
- **Bounded waiting**: If a thread attempts to acquire a lock, there is a bound on the number of times other threads can successfully acquire the lock before it does.

A Thread-Safe Bounded Queue

Locks can be used to implement a thread-safe bounded queue:

- A **bounded queue** has a fixed limit on the number of items it can contain.
- A **thread-safe** bounded queue is safe to use with multiple threads.
- The queue contains:
  - An array of items with a fixed capacity
  - Integers keeping track of the number of items added and removed
  - A lock to guard this data.
- Back index = number of items added % capacity.
- Front index = number of items removed % capacity.
- Number of items = number of items added - number of items removed.

Trying to Add an Item to the Queue

To try to add an item to the queue:

1. Set the return value to `false`.
2. Acquire the lock.
3. If the queue is full, skip to step 7.
4. Add the item provided to the queue.
5. Increment the number of items added.
6. Set the return value to `true`.
7. Release the lock.
8. Return the return value.

Trying to Remove an Item from the Queue

To try to remove an item from the queue:

1. Set the return value to `false`.
2. Acquire the lock.
3. If the queue is empty, skip to step 7
4. Copy the item from the queue to the location provided.
5. Increment the number of items removed.
6. Set the return value to `true`.
7. Release the lock.
8. Return the return value.

Condition Variables: Waiting for a Change

- In the thread-safe bounded queue example we used a polling approach to determine if it is safe to add or remove items.
- A thread could repeatedly try to add an item to a full queue until another thread removes an item.
- A thread could repeatedly try to remove an item from an empty queue until another thread adds an item.
- This approach is inefficient because the waiting thread is using processor cycles to repeatedly check a condition.

Condition Variable Definition

- A **condition variable** is a synchronization variable that lets a thread efficiently wait for a change in state that is protected by a lock.
- Condition variables provide the following operations:
  - `wait` atomically releases the lock and suspends the calling thread, putting it on a waiting list for the condition variable.
  - When the thread resumes it reacquires the lock and returns from `wait`.
  - `signal` makes one thread on the waiting list runnable.
  - `broadcast` makes all threads on the waiting list runnable.
  - All operations should only be performed while the lock is held.

Condition Variable Properties

- A condition variable must maintain the following properties:
  - It keeps no state other than a queue of waiting threads.
  - The `wait` operation atomically releases the lock.
  - If a thread releases the lock before calling `wait`, it might miss a `signal` from another thread.
  - When a waiting thread is made ready by a call to `signal` or `broadcast` it may not run immediately.
  - The condition the thread was waiting for may not hold when the thread resumes.
- Because of these properties, the `wait` operation must always be called in a loop that checks the condition being waited for.

Condition Variable Properties – Why?

Why implement condition variables this way?

- The implementation is simpler since the `wait` operation doesn't have to worry about scheduling the thread.
- Putting the `wait` operation in a loop guarantees that the condition being waited for holds when the loop exits.
- The programmer doesn't have to worry about calling `signal` or `broadcast` at the wrong time.

Thread Life Cycle Revisited

- The operations on locks and condition variables impact a thread's life cycle.
- `acquire` and `wait` may move a thread from the RUNNING state to the WAITING state.
- The thread is then moved to the waiting list for the lock or condition variable.
- `release`, `signal`, and `broadcast` may move a thread (or multiple threads) from the WAITING state to the READY state.
- The thread is then moved from the waiting list for the lock or condition variable to the ready list.

Case Study: Blocking Bounded Queue

Condition variables can be used to implement a blocking bounded queue.

- Like the polling queue, the blocking bounded queue contains an array of items with a fixed capacity, and a lock.
- In addition, there are two condition variables:
  - `itemAdded` represents the condition than an item has been added to the queue.
  - `itemRemoved` represents the condition that an item has been removed from the queue.
- Unlike the polling queue, the add and remove operations block the caller until the operation can be safely performed.

Adding an Item to the Queue

To add an item to the queue:

1. Acquire the lock.
2. Repeatedly call `wait` on `itemRemoved` until the queue is no longer full.
3. Add the item to the queue and increment the number of items added.
4. Call `signal` on `itemAdded`.
5. Release the lock.

Removing an Item from the Queue

To remove an item from the queue:

1. Acquire the lock.
2. Repeatedly call `wait` on `itemAdded` until the queue is no longer empty.
3. Remove the item from the queue and increment the number of items removed.
4. Call `signal` on `itemRemoved`.
5. Release the lock.

Object-Oriented Design Principles

Basic object-oriented design principles also apply to shared objects:

- Decompose the problem into objects.
- Define a clean interface for each object.
- Identify necessary internal state and invariants.
- Implement methods to manipulate the internal state.

Shared Object Implementation – Steps 1-3

In addition, the following steps should be performed for every shared object:

1. Add a lock.
2. Add code to acquire and release the lock.
   - Acquire the lock at the beginning of every public method.
   - Release the lock at the end of every public method.
3. Identify and add condition variables.
   - Option 1: add a condition variable for each event that a thread must wait for.
   - Option 2: use a single condition variable, and broadcast to all waiting threads.

Shared Object Implementation – Steps 4-5

4. Add loops to wait using the condition variables.
   - Returning from `wait` does not guarantee the condition holds.
   - Making no assumptions about the code calling `signal`/`broadcast` improves modularity.
5. Add `signal` and `broadcast` calls.
   - Use `signal` when any one thread waiting on the CV can make progress.
   - Use `broadcast` when multiple threads can make progress or the CV is used for multiple conditions.

Best Practices for Shared Objects

Use the following best practices when designing a shared object:

1. Follow a consistent structure.
2. Always synchronize with locks and condition variables.
3. Always acquire the lock at the beginning of a method and release it right before the return.
4. Always hold the lock when operating on a condition variable.
5. Always wait in a while loop.
6. Do not use `thread_sleep` or `thread_yield` when a thread needs to wait for a condition to hold.

Readers/Writers Locks

A **readers/writers lock (RWLock)** is a lock with the following properties:

- Any number of reader threads can be granted access to read data at the same time as long as no thread has access to write data.
- Only one writer thread can be granted access to write data at a time.

A RWLock provides the following operations:

- `startRead` blocks the caller until it is safe to read, then acquires the lock.
- `doneRead` is used by a reader to release the lock.
- `startWrite` blocks the caller until it is safe to write, then acquires the lock.
- `doneWrite` is used by a writer to release the lock.

Readers/Writers Locks – Steps 1-2

To implement a RWLock, we follow the 5-step strategy:

1. Use a lock to guard the internal state of the RWLock, which consists of variables keeping track of the following information:
   - The number of active reader threads.
   - The number of active writer threads.
   - The number of waiting reader threads.
   - The number of waiting writer threads.
2. All operations start by acquiring the lock and end by releasing the lock.

Readers/Writers Locks – First Pass

RWLock:
```
lock mutex
int activeWriters
int activeReaders
int waitingWriters
int waitingReaders
```

startRead:
```
acquire mutex




release mutex
```

doneRead:
```
acquire mutex




release mutex
```

startWrite:
```
acquire mutex




release mutex
```

doneWrite:
```
acquire mutex




release mutex
```

Readers/Writers Locks – Steps 3-5

3. Use two condition variables to make threads wait:
   - `readGo` makes a thread wait until it is safe to read.
   - `writeGo` makes a thread wait until it is safe to write.
4. Add loops to wait on condition variables in `startRead` and `startWrite`.
   - `startRead` calls `wait` on `readGo` if there are any active or waiting writers.
   - `startWrite` calls `wait` on `writeGo` if there are any active readers or writers.
5. Add code to signal/broadcast in `doneRead` and `doneWrite`.
   - `doneRead` calls `signal` on `writeGo` if there are no active readers and at least one waiting writer.
   - `doneWrite` calls `signal` on `writeGo` if there is at least one waiting writer, otherwise it calls `broadcast` on `readGo`.

Readers/Writers Locks – Second Pass

```
RWLock:
lock mutex
int activeWriters
int activeReaders
int waitingWriters
int waitingReaders
CV readGo
CV writeGo
```

```
startRead:
acquire mutex

while activeWriters > 0 or
      waitingWriters > 0
  wait on readGo


release mutex

doneRead:
acquire mutex

if activeReaders = 0 and
   waitingWriters > 0
  signal writeGo
release mutex
```

```
startWrite:
acquire mutex

while activeWriters > 0 or
      activeReaders > 0
  wait on writeGo


release mutex

doneWrite:
acquire mutex

if waitingWriters > 0
  signal writeGo
else
  broadcast readGo
release mutex
```

Readers/Writers Locks – Maintaining State

Throughout this process we also have to maintain our internal state:

- `startRead` and `startWrite` must increment the number of waiting readers and writers respectively before entering their wait loops, then decrement afterwards.
- `startRead` and `startWrite` must also increment the number of active readers and writers respectively before releasing the lock.
- `doneRead` and `doneWrite` must decrement the number of active readers and writers respectively, before checking their signaling conditions.

Readers/Writers Locks – Final Pass

RWLock:
```
lock mutex
int activeWriters
int activeReaders
int waitingWriters
int waitingReaders
CV readGo
CV writeGo
```

startRead:
```
acquire mutex
waitingReaders++
while activeWriters > 0 or
      waitingWriters > 0
  wait on readGo
waitingReaders--
activeReaders++
release mutex
```

doneRead:
```
acquire mutex
activeReaders--
if activeReaders = 0 and
   waitingWriters > 0
  signal writeGo
release mutex
```

startWrite:
```
acquire mutex
waitingWriters++
while activeWriters > 0 or
      activeReaders > 0
  wait on writeGo
waitingWriters—
activewriters++
release mutex
```

doneWrite:
```
acquire mutex
activeWriters--
if waitingWriters > 0
  signal writeGo
else
  broadcast readGo
release mutex
```

Synchronization Barriers

- Data parallel programming executes a set of threads, each one processing a different partition of a large data set.
- When all threads have completed their work, they can use the data generated by other threads in the next step in the algorithm.
- A **synchronization barrier** is a shared object used to make a set of threads wait until all of the threads in the set have completed a task.
- The barrier has one operation called `checkin` that blocks a thread until all threads in the set have called `checkin`.

Synchronization Barrier Implementation

To implement a synchronization barrer, we follow the 5-step strategy:

1. Use a lock to guard the internal state of the barrier:
   - The total number of threads in the set.
   - The number of threads that have checked in so far.
2. The `checkin` operation starts by acquiring the lock and ends by releasing it.
3. Use a condition variable called `allCheckedIn` to make each thread wait until all threads have called `checkin`.
4. Add a loop to `checkin` that checks if all threads have checked in and waits on `allCheckedIn` if they have not.
5. The last thread to call `checkin` broadcasts on `allCheckedIn`.

`checkin` must also increment the number of threads that have checked in.

FIFO Blocking Bounded Queue

- **Starvation** can occur if a thread is prevented from making progress by other threads.
- The thread-safe blocking queue we implemented does not prevent starvation if other threads remove items before a waiting thread can do so.
- Starvation can be eliminated if threads remove items in FIFO order.
- This can be implemented by maintaining a queue of condition variables, one for each removing thread.
- When a signal operation is needed it signals the condition variable at the front of the queue.

Implementing Synchronization Objects

- Implementing synchronization objects for kernel-level threads is simpler as no mode switches are required.
- Techniques for kernel-level threads can be adapted for user-level threads.
- Locks and condition variables both have state.
  - A lock must keep track of whether it is free or busy.
  - Both locks and condition variables maintain queues of waiting threads
- There are two approaches to atomically modifying this state:
  - Temporarily disable interrupts while making modifications.
  - Use an atomic read-modify-write instruction.
- Both approaches act as a **memory barrier** that guarantees all prior instructions are completed before the atomic instruction is executed.

Implementing Uniprocessor Locks

- A uniprocessor lock can be implemented by disabling interrupts.
  - The `acquire` operation disables interrupts.
  - The `release` operation enables interrupts.
  - Only useful for short critical sections in kernel code.
- Alternatively, the lock can maintain a state variable and a waiting list.
  - If the lock state is BUSY, the `acquire` operation puts the calling thread on a waiting list, sets its status to `WAITING`, switches to a thread from the ready list, and sets its status to `RUNNING`.
  - If the lock state is `FREE`, the `acquire` operation sets it to BUSY.
  - If the waiting list is empty, the release operation sets the lock state to `FREE`.
  - Otherwise, the release operation removes a thread from the waiting list, sets its status to READY, and puts it on the ready list.
  - Both operations disable interrupts for the duration of the operation.

Implementing Multiprocessor Spinlocks

- On a multiprocessor, disabling interrupts for a single processor will not affect threads running on other processors.
- A **spinlock** uses an atomic read-modify-write instruction to maintain a state variable where 0 means FREE and 1 means BUSY.
- The `test_and_set` instruction atomically reads a value from memory to a register and sets the memory value to 1.
- The `acquire` operation repeatedly executes `test_and_set` on the state variable until it reads a 0.
- The `release` instruction sets the state variable to 0 and executes a memory barrier instruction.

Implementing Multiprocessor Queueing Locks

- A multiprocessor queueing lock maintains a state variable and a waiting list, and uses a spinlock to guard them.
- The `acquire` operation does the following:
  1. Acquire the spinlock.
  2. If the lock state is BUSY, add the calling thread's TCB to the waiting list, and invoke the thread scheduler to suspend the thread, passing it the spinlock.
  3. If the lock state is FREE, set it to BUSY and release the spinlock.
- The `release` operation does the following:
  1. Acquire the spinlock.
  2. If the waiting list is empty, set the lock state to FREE.
  3. Otherwise remove a TCB from the waiting list and invoke the thread scheduler to make it ready, passing it the TCB.
  4. Release the spinlock.

Implementing the Thread Scheduler

- The scheduler uses a spinlock to guard the ready list.
- All operations begin by disabling interrupts and acquiring the spinlock and end by releasing the spinlock and enabling interrupts.
- When the scheduler suspends a thread it does the following:
  1. Release the lock passed by the calling thread.
  2. Set the calling thread's state to `WAITING`.
  3. Remove a thread from the ready list and switch to that thread.
  4. Set the newly running thread's state to `RUNNING`.
- When the scheduler makes a thread ready it does the following:
  1. Add the TCB passed by the calling thread to the ready list.
  2. Set the TCB state to `READY`.

Implementing Condition Variables

- A condition variable maintains a waiting list.
- The `wait` operation is passed a lock and does the following:
  - Add the calling thread's TCB to the waiting list.
  - Invoke the scheduler to suspend the calling thread, passing it the lock.
  - Reacquire the lock when the thread resumes.
- The `signal` operation does the following:
  - If there are no TCBs on the waiting list, do nothing and return.
  - Otherwise, remove a TCB from the waiting list and invoke that scheduler to make that thread ready.
- The `broadcast` operation performs the same steps as the signal operation, but for every TCB on the waiting list.

Application-Level Synchronization

- Kernel threads keep all synchronization data and code in kernel memory and run in kernel mode.
- Kernel-managed user threads are backed by kernel threads while the kernel manages the ready list.
  - If the synchronization objects are in kernel space, the operations can be implemented as system calls.
  - If some data is kept in user space, a mode switch can be avoided when acquiring a free lock or signaling a condition variable with no waiting threads.
- User-managed threads use kernel threads as virtual processors and keep all code and data in user space.
  - User-level thread packages can not disable interrupts.
  - The process running the threads can disable upcalls while managing the ready list.

Semaphores

- A **semaphore** is a synchronization object that keeps track of a nonnegative integer value.
- A semaphore can be created with any initial nonnegative value.
- The down operation does the following:
  - Suspend the calling thread if the value is zero.
  - Atomically decrement the value by one.
- The up operation does the following:
  - Atomically increments the value by one.
  - Enables one thread if there are any threads waiting.

Using Semaphores

- A semaphore can be used for mutual exclusion as follows:
  - Set the initial value to one.
  - Use down to acquire the semaphore.
  - Use up to release the semaphore.
- A semaphore can be used for general waiting as follows:
  - Set the initial value to zero.
  - Use down to wait on the semaphore.
  - Use up to signal a waiting thread to resume.

Semaphores and Condition Variables

- Semaphores differ from condition variables in several ways:
  - Semaphores maintain a state, while condition variables do not.
  - down does not atomically release a lock while suspending a thread or reacquire a lock when the thread resumes.
  - If no threads are waiting on the semaphore, up will change the state from zero to one, which means the next thread to call down will not be blocked.
- Semaphores can be used to implement condition variables as follows:
  - The condition variable maintains a queue of semaphores.
  - `wait` creates a semaphore with a value of zero, adds it to the queue, releases the lock, calls down on the semaphore, then acquires the lock.
  - If the queue is not empty `signal` removes a semaphore from the queue and calls up on the semaphore, otherwise it does nothing.